

Glitch Grow — Google Ads API Design Document

Glitch Executor Labs · grow.glitchexecutor.com · Version 1.0

1. Product overview

Glitch Grow is an AI-powered marketing-operations platform for direct-to-consumer e-commerce brands. It is operated by Glitch Executor Labs as an internal tool for managing our book of client accounts — not a multi-tenant SaaS. Each client explicitly links their Google Ads account to our Manager (MCC) account before we provision any automation for them.

The platform unifies four signal domains into a single operator workflow: search (Google Search Console + PageSpeed Insights + Natural Language), ads (Google Ads + Meta Ads + Amazon Attribution), shop data (Shopify Admin API), and on-site behaviour (GA4 + Meta Pixel reconciliation). The Google Ads API is the authoritative source for everything we do on the ads side.

2. How we use the Google Ads API

2.1 Read paths

- Campaign, ad-group, ad, asset, and keyword performance reports ingested daily into our internal analytics store. Used to build operator dashboards and feed AI models that generate recommendations (bid, budget, creative rotation).
- Conversion-action metadata — names, categories, goal status, counting rules — pulled to audit configuration drift across client accounts and flag discrepancies.
- Audience and customer-match list metadata for lifecycle reporting. We do not re-upload end-user PII to the API; we rely on the client-side Pixel + Shopify's native Customer Match integration for audience construction.

2.2 Write paths

- Create and maintain conversion actions on client accounts at scale — wiring purchase events from Shopify, Shiprocket Fastr, and FlexyPe checkouts to the right Google Ads goals with consistent naming and value/currency fields.
- Upload offline conversions (Amazon Attribution click matches and COD-confirmed Shopify orders) to close the cross-channel attribution loop that the standard pixel misses.
- Apply a small, bounded set of campaign-level optimisations (budget nudges, negative keyword additions, asset group enablement) that our operators review and approve before any API mutation is sent.

2.3 What we do NOT do

- No unattended autonomous bidding or account-structure changes. Every mutating API call either originates from an operator action in our admin UI or runs on a scheduled job whose change-set is audit-logged and reviewable.
- No exposure of the Google Ads API to end-users, clients, or third parties. Clients receive read-only dashboards built from our cached data; they never call the API directly through our tool.
- No reselling, white-labelling, or syndication of API access.

- No retention of raw API responses beyond what is needed for reporting and change-set auditing.

3. System architecture

Backend runs on Node.js (React Router, Shopify app) plus a sibling Python pipeline for long-running signal pulls. Both services run on a dedicated VM under Glitch Executor Labs' control. Data is stored in PostgreSQL with pgvector for embeddings used by the AI recommendation models.

Credentials are scoped to a single Google Cloud service account with narrowly-scoped API access. The Google Ads developer token and Manager customer id are loaded from an environment file that never leaves the VM and is not committed to version control. We rotate credentials on a quarterly cadence.

Layer	Role	Google Ads touch-points
Node admin app	Operator UI, approvals, audit log	Orchestrates approved mutations only
Python agent	Scheduled signal pulls, AI analysis	Read-only reports (searchStream)
PostgreSQL	Cached metrics, audit trail, AI embeddings	Stores materialised views + lineage
Service account	Single identity for all API calls	Linked to MCC with user access

4. Estimated API usage

Endpoint / Resource	Frequency	Requests / day
campaign, ad_group, ad, asset reports	Daily batched pulls	~1,000
conversion_action metadata	Twice daily audit	~40
conversion upload (offline)	Event-driven, batched	~200
conversion_action mutate (create/update)	Operator-approved	<50
customer + linked customer_client	Weekly	~20
Total (current 4-brand scale)	&mdash;	~1,300
Projected (20-brand scale, 12 months)	&mdash;	~7,000

Report pulls are chunked, paginated, and cached aggressively on our side (see § 5). We do not re-query data we already have; incremental pulls only fetch rows changed since the last successful fetch.

5. Required Minimum Functionality (RMF) compliance

RMF requirement	How Glitch Grow meets it
Accurate reporting	All report data is surfaced with source citation (report name, date range, last-refreshed t
Campaign management (create/edit/delete)	Operator in UI supports the full lifecycle for the campaign types listed in §2. Operato
Ad group / ad / keyword management	Supported via the same preview-first pattern. Keyword uploads are diff-checked against
Negative keywords and exclusions	Supported at campaign and account level with operator approval.
Conversion tracking	Primary use-case (see §2.2). Conversion actions are created consistently across c
Budget management	Budget changes originate from operator action or AI-flagged recommendations that an c
Reporting — format + customisation	Operators choose date range, segmentation, and metrics in the admin UI. Reports can l
Destination of advertiser data	Stored in our own PostgreSQL only. Not sold, shared, or syndicated to any third party.

6. Security and data handling

- Service account credentials and developer token are stored in restricted .env files on the production VM only (0600 mode, support user only). Never committed to version control; never exposed to client browsers or third parties.
- Transport: all API calls are made over TLS to googleads.googleapis.com. No intermediate proxies or request logging services see raw credentials.
- Access model: one service account per environment (staging / prod). Rotated quarterly and immediately on any team-member departure.
- Data retention: report data is cached for 90 days at raw granularity, then rolled into daily aggregates. Conversion audit logs are retained for 13 months to satisfy Google Ads policy-review requests.
- PII handling: we do not transmit advertiser's end-user PII through the API. Customer-match and audience features use Shopify's native Google Ads integration, which performs hashing on its side.

7. Why Basic access is needed

All client accounts linked to our MCC are production accounts with live ad spend. Test-account access does not allow us to read or write against these accounts, which blocks our primary use-cases — conversion management, reporting, and offline-conversion upload for attribution reconciliation.

Volume and throughput under Basic access (15,000 operations per day per developer token) are more than sufficient for our current 4-brand scale and projected 20-brand scale over the next twelve months. We do not currently need Standard access.

8. Contact

Tooling URL: <https://grow.glitchexecutor.com>

Technical contact: dev@glitchexecutor.com

Organisation: Glitch Executor Labs (operator of the Glitch Grow marketing-operations platform).